INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

GRC102 Security Governance Simulation Assessment Brief

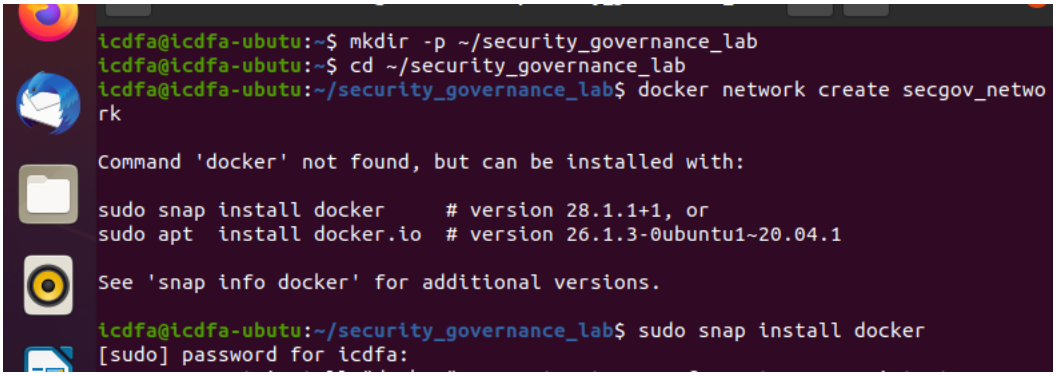
Practical Lab 8 Monitoring Auditing Controls and Executive Reporting

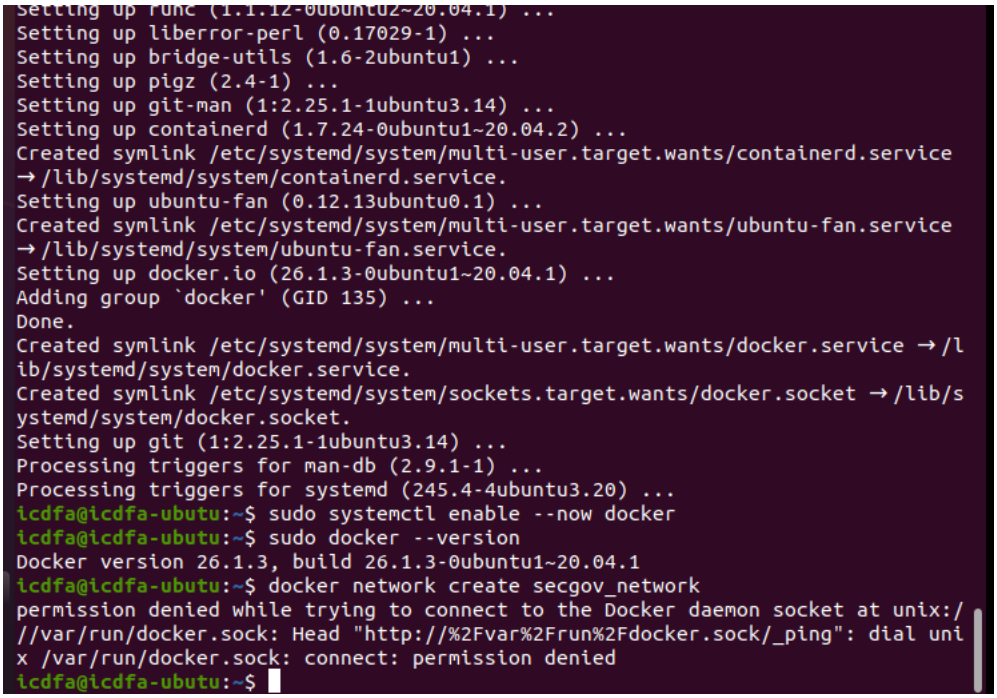
RABIATU UMAR AHMAD

C11/26/CGRCE/17569

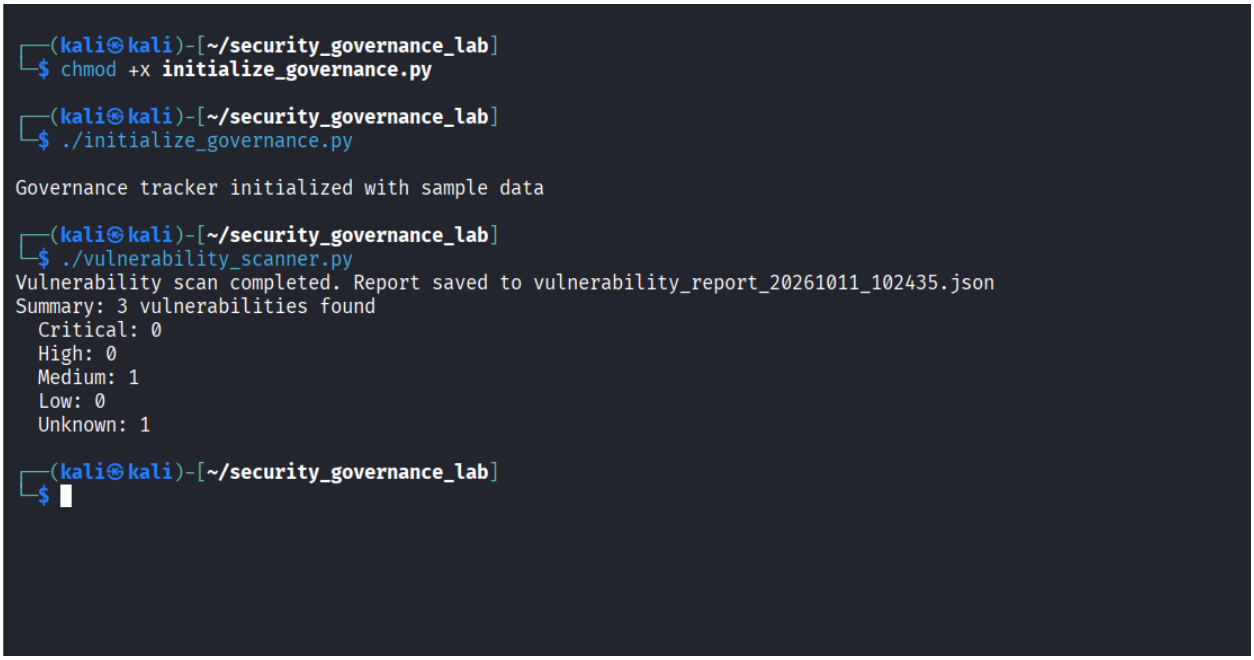
COHORT 11

11th October, 2026





Started using the icdfa kali but couldn’t copy and paste the files.



The security governance environment reflects some of the conditions that led to the Equifax data breach. In the Equifax incident, attackers exploited a known vulnerability in the Apache Struts web application framework that had not been patched, allowing them to gain unauthorized access to sensitive personal information. Similarly, this simulated environment demonstrates how weaknesses in system configuration, outdated software, poor patch management, and inadequate security monitoring can expose an organization to cyberattacks.

Part 2: Analyzing Security Governance Failures


Session Actions Edit View Help
attack_log = {
"timestamp": datetime.datetime.now().isoformat(),
"attack_type": "Struts S2-045 Exploitation",
"target": "web_server",
"success": True,
"data_accessed": "customer_data database",
"records_affected": 3
}
with open("attack_log.json", "w") as f:
json.dump(attack_log, f, indent=2)
print("Attack simulation completed. Log saved to attack_log.json")
return True
else:
print("System is not vulnerable to Struts S2-045 (CVE-2017-5638)")
return False
except Exception as e:
print(f"Error simulating attack: {e}")
return False
def main():
simulate_struts_attack()
if __name__ == "__main__":
main()
EOF
(kali㉿kali)-[~/security_governance_lab]
$ chmod +x simulate_attack.py
(kali㉿kali)-[~/security_governance_lab]
$

```
    }

    # Count failures by category
    for failure in failures:
        category = failure["category"]
        if category in report["summary"]["categories"]:
            report["summary"]["categories"][category] += 1
        else:
            report["summary"]["categories"][category] = 1

    return report

def main():
    failures = analyze_governance_failures()
    report = generate_report(failures)

    report_file = f"governance_failure_report_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
    with open(report_file, "w") as f:
        json.dump(report, f, indent=2)

    print(f"Governance failure analysis completed. Report saved to {report_file}")
    print(f"Summary: {report['summary']['total_failures']} governance failures identified")

    for category, count in report["summary"]["categories"].items():
        print(f"  {category}: {count}")

if __name__ == "__main__":
    main()
heredoc> EOF

┌──(kali㉿kali)-[~/security_governance_lab]
└─$ chmod +x analyze_governance_failures.py

┌──(kali㉿kali)-[~/security_governance_lab]
└─$
```

Running the attack simulation

```
Session  Actions  Edit  View  Help
            report["summary"]["categories"][category] += 1
        else:
            report["summary"]["categories"][category] = 1

    return report

def main():
    failures = analyze_governance_failures()
    report = generate_report(failures)

    report_file = f"governance_failure_report_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
    with open(report_file, "w") as f:
        json.dump(report, f, indent=2)

    print(f"Governance failure analysis completed. Report saved to {report_file}")
    print(f"Summary: {report['summary']['total_failures']} governance failures identified")

    for category, count in report["summary"]["categories"].items():
        print(f"  {category}: {count}")

if __name__ == "__main__":
    main()
heredoc> EOF

┌──(kali㉿kali)-[~/security_governance_lab]
└─$ chmod +x analyze_governance_failures.py

┌──(kali㉿kali)-[~/security_governance_lab]
└─$ ./simulate_attack.py
Simulating attack on vulnerable Struts application...
Error simulating attack: HTTPConnectionPool(host='web_server', port=8080): Max retries exceeded with url: / (Caused by NameResolutionEr
='web_server', port=8080): Failed to resolve 'web_server' ([Errno -2] Name or service not known)"))

┌──(kali㉿kali)-[~/security_governance_lab]
└─$
```

Analyze the governance failures

```
┌──(kali㉿kali)-[~/security_governance_lab]
└─$ ./analyze_governance_failures.py

No attack log found
Governance failure analysis completed. Report saved to governance_failure_report_20261011_104352.json
Summary: 4 governance failures identified
  Policy Implementation: 1
  Metrics and Measurement: 1
  Security Architecture: 1
  Authentication and Access Control: 1

┌──(kali㉿kali)-[~/security_governance_lab]
└─$
```

```
5. **Policy Implementation Failure**: Security policies were not effectively implemented or enforced.

## Comparison with Simulated Environment

| Governance Failure | Equifax | Simulated Environment | Similarity |
|---|---|---|---|
| Patch Management | Failed to patch Apache Struts vulnerability for months | Unpatched Apache Struts vulnerability in web server | High
| Security Monitoring | Failed to detect breach for 76 days | No real-time monitoring to detect attack | High |
| Network Segmentation | Insufficient segmentation allowed lateral movement | Web server can directly access database | High |
| Authentication | Weak credentials and access controls | Default credentials on database | Medium |
| Policy Implementation | Policies not effectively implemented | Patch and vulnerability management policies exist but not enforced | Hi
| Metrics and Measurement | Ineffective security metrics | Metrics not driving security improvements | Medium |

## Lessons Learned

1. **Effective Patch Management is Critical**: Both Equifax and our simulation demonstrate that failing to patch known vulnerabilities c

2. **Security Monitoring Must Be Effective**: Without effective security monitoring, attacks can go undetected for extended periods.

3. **Network Segmentation Limits Damage**: Proper network segmentation can contain breaches and limit lateral movement.

4. **Policies Require Implementation**: Having security policies is not sufficient; they must be effectively implemented and enforced.

5. **Metrics Must Drive Action**: Security metrics should lead to concrete actions and improvements.

## Conclusion

The simulated environment successfully recreates many of the security governance failures that contributed to the Equifax data breach. B
res, we can better understand the importance of effective security governance and develop strategies to prevent similar incidents in the
EOF

┌──(kali㉿kali)-[~/security_governance_lab]
```

One key governance lesson learned from the Equifax breach is that organizations must establish and enforce strong cybersecurity policies and procedures. Regular vulnerability assessments, timely security patching, continuous monitoring, and effective access controls are essential for protecting sensitive information. Organizations should also clearly assign security responsibilities and ensure that identified vulnerabilities are addressed promptly.

Part 3: Implementing Security Governance Controls

Create a script to implement security governance controls: